Encapsulation (1)

Un `Ingredient` (culinaire) a un nom et un certain pouvoir calorique (par 100 grammes). Un `Plat` est composé de divers ingrédients intervenants chacun pour une certaine quantité (comptée en grammes). Les ingrédients sont ajoutés au plat au fur et à mesure. Pour chacun, on donne la quantité. Vous pouvez considérer que personne n'ajoutera jamais deux fois le même ingrédient au même plat. Le plat mémorise les ingrédients et leur quantité respective dans un `Map<Ingredient, Integer>`.
Les classes suivantes compilent et la classe `Cuisine` s'exécute sans exception.

```
class Ingredient {
    String nom;
    int calories;
    Ingredient(String nom, int calories) {
        this.nom = nom;
        this.calories = calories;
    }
}

class Plat {
    String nom;
    Map<Ingredient, Integer> ingredients = new HashMap<>();
}

public class Cuisine {
    public static void main(String[] args) {
        Ingredient tomate = new Ingredient("Tomate", 18);
        Ingredient mozzarella = new Ingredient("Mozzarella", 280);
        Plat tomoza = new Plat(); tomoza.nom = "Tomate-Mozzarella";
        tomoza.ingredients.put(tomate, 300);
        tomoza.ingredients.put(mozzarella, 200);

        System.out.print("Le plat " + tomoza.nom + " se compose de: ");
        for(Ingredient i: tomoza.ingredients.keySet())
            System.out.print(i.nom + " ");
        int calories = 0;
        for(Ingredient i: tomoza.ingredients.keySet())
            calories += i.calories * tomoza.ingredients.get(i) / 100;
        System.out.println("\nSa valeur calorique est: " + calories);
    }
}
```

Question 1 : Sans changer le comportement du programme, corrigez les classes (`Cuisine` y compris) afin de respecter les règles suivantes. **La page suivante reproduit les mêmes classes en vous laissant de la place pour les corriger**.

1. Une fois fixés, le nom et les calories d'un ingrédient ne peuvent plus être changés.
2. Les calories ne peuvent être inférieures à 0.
3. Le nom d'un plat ne peut être modifié.
4. Les classes doivent respecter les conventions Java, les règles de bonnes pratiques d'encapsulation, de couplage et de cohésion. En particulier, la loi de Demeter (« *ne pas parler aux étrangers* ») doit être respectée.

Encapsulation (2)

```
class Ingredient {









}

class Plat {









}

public class Cuisine {
  public static void main(String[] args) {









  }
}
```

Jeu de piste

Soit le code suivant (ce code compile) :

```
class A {
    int i = 1;
    public A() {
        System.out.println("A : " + i);
    }
    public void setI(int i) {
        this.i = 2 * i;
    }
}

class B extends A {
    public B() {
        setI(2);
        System.out.println("B : " + i);
    }
    public void setI(int i) {
        this.i = 3 * i;
    }
}

class C extends B {
    public C() {
        System.out.println("C : " + i);
    }
    public void setI(int i) {
        this.i = 4 * i;
    }
}
```

Question 2 : Quel est le résultat de l'exécution du code suivant ?

```
new B();
```

Question 3 : Quel est le résultat de l'exécution du code suivant ?

```
new C();
```

Itérateurs

Soit le code suivant qui itère sur une étendue de `Dates` (la classe `Date` est celle vue au cours).

```
public class Test {
    public static void main(String[] args) {
        Date deb = new Date(30, 12, 2017);
        Date fin = new Date(2, 1, 2018);
        for (Date d : new DateRange(deb, fin))
            System.out.println(d);
    }
}

class DateRange implements Iterable<Date> {
    Date deb, fin;

    public DateRange(Date deb, Date fin) {
        this.deb = deb;
        this.fin = fin;
    }
    @Override public Iterator<Date> iterator() {
        return new DateIterator(this);
    }
}

class DateIterator implements Iterator<Date> {
    DateRange dr;
    Date cour;

    DateIterator(DateRange dr) {
        this.dr = dr;
        cour = dr.deb;
    }
    @Override public boolean hasNext() {
        return cour.compareTo(dr.fin) <= 0;
    }
    @Override public Date next() {
        Date res = cour;
        cour.increment();
        return res;
    }
}
```

```
// Devrait afficher...
// Samedi 30 Décembre 2017
// Dimanche 31 Décembre 2017
// Lundi 1 Janvier 2018
// Mardi 2 Janvier 2018

// mais affiche...
// Dimanche 31 Décembre 2017
// Lundi 1 Janvier 2018
// Mardi 2 Janvier 2018
// Mercredi 3 Janvier 2018
```

Question 4 Une erreur classique s'y est glissée et le programme compile mais ne se comporte pas correctement. Quelle est cette erreur ? Corrigez-la.